

06: System Design: Cloud GPU Services

The system design interview for a GPU cloud role usually asks for a variation of one problem:

"Design a cloud GPU service where users can create, connect to, and monitor GPU instances or GPU-accelerated workloads."

This chapter works through that problem in the order you would present it in a 45-minute interview: requirements, architecture, the components that deserve a closer look, scaling, failure handling, and security. It ends with an outline you can use on a whiteboard and a set of design positions you should be able to defend.

This chapter covers

- Turning an open-ended prompt into functional and non-functional requirements
- A control plane and data plane architecture for GPU workloads
- An asynchronous, idempotent API for long-running create operations
- Preventing quota races with reservations
- GPU sharing, health monitoring, and placement
- Failure modes, scaling limits, and multi-tenant security

Contents

Step 1: Clarify the requirements.....	2
Functional requirements	2
Non-functional requirements.....	2
Step 2: High-level architecture	3
Step 3: Examine the key components	4
The API, SDK, and CLI.....	4
Quota and admission	4
The provisioning workflow.....	5
Storage and state.....	5
GPU scheduling and utilization	5
Step 4: Scale and performance	6
Step 5: Availability and failure modes.....	7
Step 6: Security and multi-tenancy	8
A whiteboard outline	9
Design positions to defend	10
Summary	11

Step 1: Clarify the requirements

Spend the first five to ten minutes on requirements. Ask what kind of workload the service runs, because the answer changes the design: virtual machines with SSH access, batch jobs, and inference endpoints have different lifecycles. State your assumptions aloud and write them down.

Functional requirements

- Users can create, list, inspect, and delete GPU workloads: instances, jobs, or inference endpoints.
- Users choose the GPU model and count, region or zone, image, storage, and network settings.
- Users connect to workloads through SSH, Jupyter, an HTTP API, or gRPC.
- Users can see workload health, GPU metrics, logs, and cost.
- Administrators manage quotas, access control, node pools, and maintenance windows.
- Workloads recover automatically when a node fails.

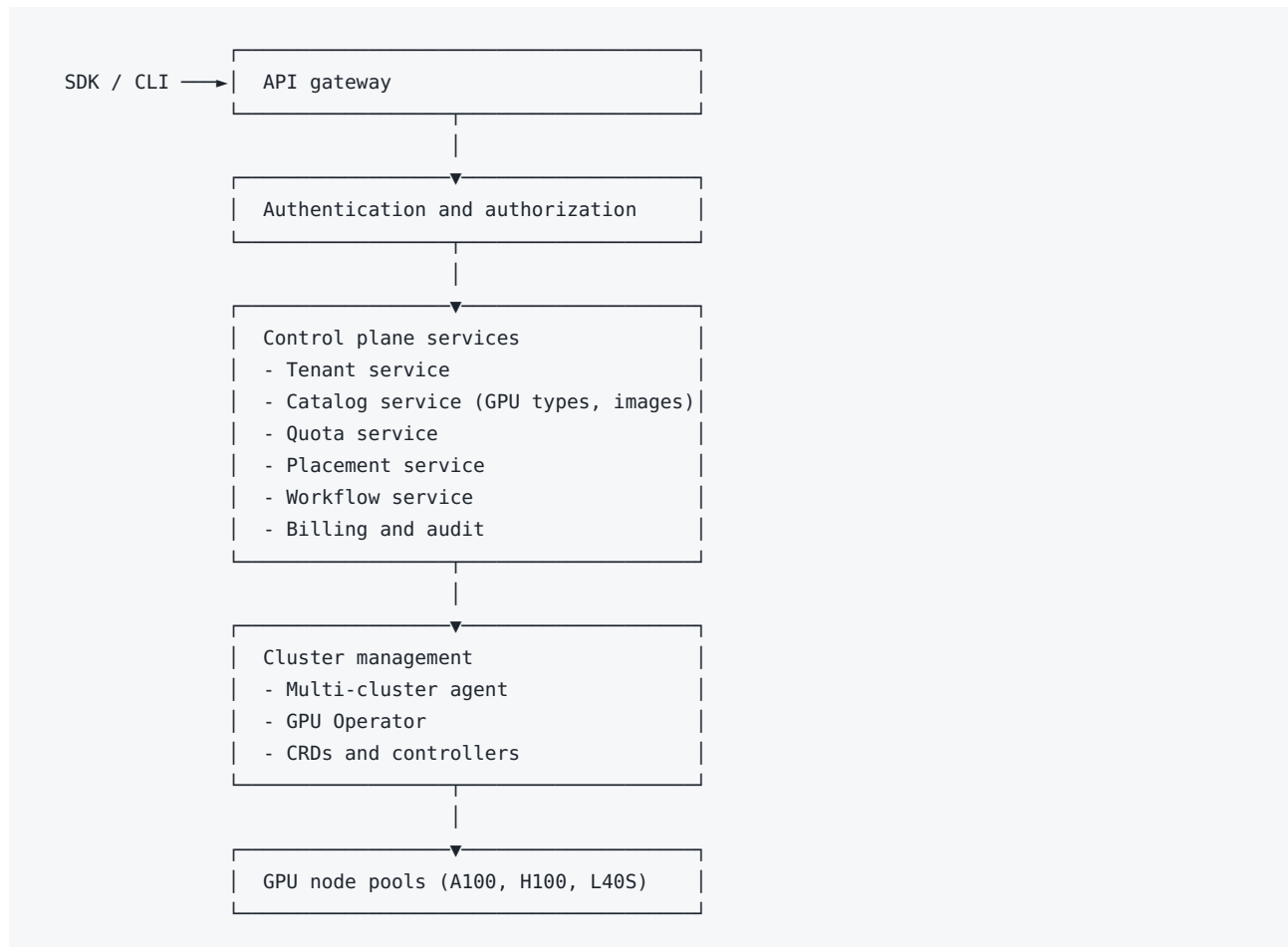
Non-functional requirements

Give numbers where you can. They drive the later design decisions.

Requirement	Target and notes
Multi-tenancy	Tenants cannot see or affect each other's workloads, data, or network traffic
Availability	The control plane runs across several availability zones. Running workloads continue if the control plane is unavailable
Create latency	p95 under 30 seconds for a container workload. Image pulls, driver initialization, and firmware set the lower bound
Scale	Thousands of concurrent create requests and millions of API calls per day
Security	TLS for all traffic, mutual TLS between services, identity-based access control, network policy, image scanning
Cost efficiency	GPU sharing (MIG, time-slicing), spot capacity, and reclaiming idle workloads
Observability	Metrics, logs, traces, and alerts for both the control plane and the workloads

Step 2: High-level architecture

Separate the system into a **control plane**, which accepts requests and decides what should run, and a **data plane**, which runs the workloads on GPU nodes.



Each control plane service has one responsibility:

Service	Responsibility
Tenant	Organizations, projects, membership, and the mapping from projects to namespaces
Catalog	Available GPU models, regions, images, and prices
Quota	Limits per project, region, and GPU model, and the reservations against them
Placement	Choosing a cluster, zone, and node pool with capacity for the requested GPU type
Workflow	Running the multi-step create and delete processes, with retries and compensation
Health	Watching workload status, node conditions, and GPU metrics
Billing and audit	Recording usage from metering events and recording who did what

Step 3: Examine the key components

Choose two or three components to discuss in depth. The interviewer may direct you; if not, the API, quota, and provisioning workflow are good choices because they contain the most interesting failure cases.

The API, SDK, and CLI

Creating a GPU workload takes seconds to minutes, so the API must not hold a request open until it finishes. Use an **asynchronous** pattern:

1. The client sends `POST /v1/gpu-workloads` with an Idempotency-Key header.
2. The server validates the request, records it, and returns `202 Accepted` with the workload ID and a `Location` header.
3. The client polls `GET /v1/gpu-workloads/{id}` for the status, or receives a webhook when the status changes.
4. `DELETE /v1/gpu-workloads/{id}` starts deletion, which is also asynchronous.

The **idempotency key** makes retries safe. If the client times out and sends the same request again, the server finds the key and returns the original workload instead of creating a second one. Without it, a network timeout can create duplicate GPU workloads, and the customer pays for both.

Generate the SDKs from an OpenAPI specification so that every language exposes the same types. Keep the CLI thin: it authenticates, calls the API through the SDK, prints a table or JSON, and exits with a meaningful status code.

Quota and admission

Before creating a workload, the service checks that the request fits within the project's quota. Quotas can apply at several levels: per project, per region, and per GPU model.

A simple check followed by a create has a race. Two requests that each ask for 4 GPUs against a remaining quota of 6 can both pass the check before either one is recorded.

Prevent this with a **reservation**:

1. The request creates a reservation, which atomically decrements the available quota in a database transaction.
2. The provisioning workflow converts the reservation into an allocation when the workload is created.
3. A reservation that is not used within a timeout expires and returns its quota.

Kubernetes ResourceQuota objects limit usage inside each cluster namespace, but the control plane still needs its own quota system: it spans many clusters, and billing depends on it.

The provisioning workflow

A create request passes through these steps:

1. Validate the request.
2. Reserve quota.
3. Choose a cluster and node pool with capacity for the requested GPU type.
4. Create a `GpuWorkload` custom resource in that cluster, or a `Deployment` directly.
5. The operator in the cluster creates the `Deployment` and `Service` and reports progress in the resource's status.
6. Wait until the Pods are running and ready.
7. Mark the workload `Ready` and start billing.

Every step can fail or time out. Make each step idempotent so the workflow can retry it, and define a **compensating action** for each step so that a failed create releases what it acquired, such as the quota reservation and any partially created Kubernetes objects. A workflow engine such as Temporal, or a state machine stored in the database, keeps track of which steps have completed.

Storage and state

Data	Store	Reason
Tenants, quotas, catalog, reservations	Relational database (PostgreSQL)	Transactions for quota accounting, and relational queries
Live workload state	Kubernetes API in each cluster	The operator already maintains it
Logs and artifacts	Object storage	Large, append-only data
Frequently read data	Cache (Redis)	Reduces database load; never the source of truth
Events for billing and notifications	Transactional outbox and event bus	A state change and its event are recorded in the same transaction, so no event is lost

GPU scheduling and utilization

- **Capacity.** Each GPU node advertises `nvidia.com/gpu`. The NVIDIA device plugin registers the GPUs with the kubelet, and the Kubernetes scheduler assigns them to Pods.
- **Sharing.** MIG divides a supported GPU into isolated instances with dedicated memory. Time-slicing lets several workloads share a GPU without memory isolation. Offer them as different products, because their isolation guarantees differ.
- **Metrics.** `dcpm-exporter` publishes GPU utilization, memory, temperature, power, and error counters to Prometheus.
- **Health.** Watch for driver failures, Xid errors, ECC errors, and thermal or power problems. Cordon nodes that report hardware faults and drain their workloads.

Step 4: Scale and performance

- **API gateway.** Global routing, rate limiting per tenant, and authentication at the edge.
- **Control plane services.** Keep them stateless so they scale horizontally.
- **Event queues.** Decouple the API from provisioning, so a spike in requests queues work instead of overloading the clusters.
- **Database.** Partition data by region or tenant, and use read replicas for list and status queries.
- **Clusters.** Use many clusters of moderate size, managed through an agent in each cluster, instead of one very large cluster. This limits the effect of a cluster failure and avoids control plane scaling limits.
- **Create latency.** Most of the time goes to pulling images, initializing the driver, and starting the Pod. Pre-pull common base images on nodes, cache images close to clusters, and keep warm capacity for popular GPU types.

Step 5: Availability and failure modes

Discuss what happens when each part fails:

Failure	Effect	Mitigation
A region fails	Workloads in that region stop	Replicate control plane data to another region, and let customers deploy across regions
A node fails	Its workloads stop	Kubernetes reschedules Pods; the cluster autoscaler replaces the node
The control plane API is unavailable	New requests fail	Clients retry with exponential backoff; running workloads are unaffected
An operator crashes	Changes are not reconciled	Existing workloads keep running; the operator reconciles everything when it restarts
A partial create	For example, the Service exists but the Pods are pending	The workload status reports each condition, and the workflow retries or compensates

Step 6: Security and multi-tenancy

- A namespace for each tenant or project.
- A default-deny NetworkPolicy for each tenant namespace.
- Role-based access control with least privilege, for users and for service accounts.
- Pod Security Standards to prevent privileged containers in tenant namespaces.
- Secrets in a secret manager, injected at run time.
- Signed images and verification of their provenance before admission.
- Audit logs for every API call that changes state.
- Mutual TLS between control plane services.

For workloads that need stronger isolation than namespaces provide, run them in dedicated node pools or in virtual machines.

A whiteboard outline

Use this order to structure your answer:

1. Functional and non-functional requirements
2. The API: resources and asynchronous operations
3. A block diagram of the control plane and data plane
4. The data model
5. The provisioning workflow
6. Kubernetes integration and the operator
7. Failure handling
8. Metrics and alerting
9. Scaling limits
10. Trade-offs and next steps

Design positions to defend

Be prepared to explain each of these positions:

- **The API is declarative.** A client states the workload it wants, and controllers work to make the actual state match. The API does not execute a sequence of commands.
- **The workload lifecycle is a state machine.** A workload moves through Pending, Provisioning, Running, Stopping, and Terminated, and each transition is explicit.
- **Retries and idempotency belong in both the SDK and the server.** The SDK retries with backoff, and the server uses idempotency keys so that retries are safe.
- **Operators reconcile the cluster to the API object.** Health checks update the object's status, and the control plane reads that status.
- **Node-level GPU management is already solved.** The device plugin and the GPU Operator manage drivers and devices on each node. The service adds tenant-level orchestration on top.

Summary

- Start with requirements and numbers, because they determine the design.
- Separate the control plane, which decides, from the data plane, which runs workloads.
- Long-running operations need an asynchronous API with idempotency keys.
- Reservations prevent quota races between concurrent requests.
- Design every workflow step to be retried and compensated.
- Plan for each failure explicitly, and keep running workloads independent of the control plane.

Source: [06-system-design-cloud-gpu.md](#) · [PDF](#) · [All study guides](#)